

Práctica 4: WHILE y codificación

Miguel Ángel Dorado Maldonado

2 de junio de 2026

1. Actividad 1: suma de tres valores

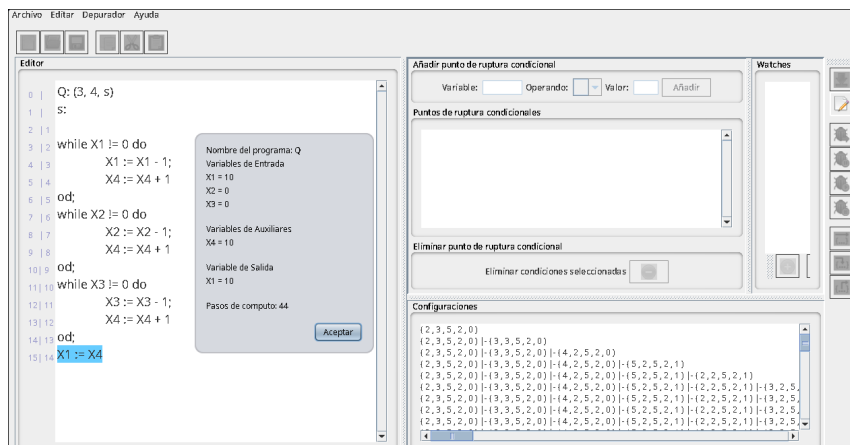
Se pide implementar un programa WHILE que calcule la suma de tres valores usando una variable auxiliar que acumule el resultado.

La función que queremos calcular es:

$$F_Q(x_1, x_2, x_3) = x_1 + x_2 + x_3$$

Usamos la variable auxiliar X_4 como acumulador. El programa va vaciando X_1 , X_2 y X_3 , y por cada unidad que resta suma una unidad a X_4 . Al final copia el resultado acumulado en X_1 .

1.1. Programa WHILE



```
from python.whilanguage import *
from python.whilanguage.encoding import *
```

```
Q = """(3,
while X1 != 0 do
    X1 := X1 - 1;
    X4 := X4 + 1
od;
while X2 != 0 do
    X2 := X2 - 1;
    X4 := X4 + 1
od;
while X3 != 0 do
    X3 := X3 - 1;
    X4 := X4 + 1
od;
X1 := X4)"""
```

```
print("TQ", t_steps(Q, [3, 5, 2]))
print("Res:", f_function(Q, [3, 5, 2]))
```

1.2. Resultado

La entrada usada es:

$$(3, 5, 2)$$

Por tanto, el resultado esperado es:

$$3 + 5 + 2 = 10$$

La ejecución en Python devuelve:

```
TQ 44  
Res: 10
```

Luego:

$$F_Q(3, 5, 2) = 10$$

$$T_Q(3, 5, 2) = 44$$

El resultado coincide con la suma esperada.

2. Actividad 2: función diverge

Se pide crear el programa WHILE más sencillo que calcule la función **diverge** con cero argumentos y calcular la codificación de su código.

La función **diverge** es una función que no termina nunca. Es decir, para su única entrada posible, ya que tiene cero argumentos, el programa no alcanza una configuración final.

2.1. Programa WHILE

Como las variables empiezan inicializadas a cero, primero hacemos que X_1 valga 1. Después usamos un bucle que se ejecuta mientras $X_1 \neq 0$. Dentro del bucle incrementamos X_1 , así que nunca vuelve a valer cero.

```
D = "X1:=X1+1; while X1!=0 do X1:=X1+1 od"
```

El programa completo tiene cero variables de entrada:

$$(0, D)$$

2.2. Explicación

Inicialmente:

$$X_1 = 0$$

Después de ejecutar la primera instrucción:

$$X_1 = 1$$

A partir de ahí se entra en el bucle:

```
while X1 != 0 do  
  X1 := X1 + 1  
od
```

Como X_1 aumenta en cada iteración, nunca se hace cero. Por tanto, el programa no termina.

2.3. Codificación

Con la función `while2n`, la codificación del programa completo se obtiene así:

```
D = "X1:=X1+1; while X1!=0 do X1:=X1+1 od"

print(while2n(0, D))
```

La función `while2n` codifica el programa completo, incluyendo el número de variables de entrada. En este caso, el número de entradas es 0.

La codificación obtenida es:

139273919255627

Por tanto:

$$\ulcorner(0, D)\urcorner = 139273919255627$$

Si se codifica solamente el código, sin incluir el número de variables de entrada, se usa `code2n(D)`. En este caso:

`code2n(D) = 16689751`

3. Actividad 3: enumerar los k primeros vectores de $\mathbb{N}^*$

Se pide crear una función de Python que enumere los k primeros vectores de $\mathbb{N}^*$. El conjunto $\mathbb{N}^*$ es el conjunto de todos los vectores finitos de números naturales:

$$\mathbb{N}^* = \bigcup_{n \geq 0} \mathbb{N}^n$$

Incluye vectores de cualquier longitud, incluido el vector vacío:

$$[], [0], [1], [0, 0], [2], \dots$$

Para enumerarlos usamos la función `godeldecoding`. Esta función decodifica cada número natural como un vector de $\mathbb{N}^*$.

3.1. Código

```
def enumera_vectores(k):
    vectores = []

    for n in range(k):
        vectores.append(godeldecoding(n))

    return vectores

print(enumera_vectores(10))
```

3.2. Resultado

La salida para $k = 10$ es:

```
[[], [0], [0, 0], [1], [0, 0, 0], [1, 0], [2],
 [0, 0, 0, 0], [1, 0, 0], [0, 1]]
```

3.3. Explicación

La función recorre los números naturales:

$$0, 1, 2, 3, 4, \dots$$

A cada número n se le aplica `godeldecoding(n)`. Como la codificación de Gödel establece una biyección entre $\mathbb{N}$ y $\mathbb{N}^*$, al recorrer los naturales se obtienen todos los vectores finitos de naturales.

Por tanto, la función `enumera_vectores(k)` devuelve los k primeros vectores de $\mathbb{N}^*$ según el orden inducido por la decodificación de Gödel.

4. Actividad 4: enumerar los k primeros programas WHILE

Se pide crear una función de Python que enumere los k primeros programas WHILE.

La idea es similar a la actividad anterior. Como los programas WHILE pueden codificarse mediante números naturales, podemos recorrer los números naturales y decodificar cada uno como programa WHILE.

Para ello usamos la función `n2while`.

4.1. Código

```
def enumera_programas_while(k):
    programas = []

    for n in range(k):
        programas.append(n2while(n))

    return programas

programas = enumera_programas_while(10)

for i, programa in enumerate(programas):
    print(i, programa)
```

4.2. Resultado

La salida para $k = 10$ es:

```
0 (0, X10)
1 (1, X10)
2 (0, X10; X10)
3 (2, X10)
4 (1, X10; X10)
5 (0, X1X1)
6 (3, X10)
7 (2, X10; X10)
8 (1, X1X1)
9 (0, X10; X10; X10)
```

4.3. Explicación

La función recorre los números:

$$0, 1, 2, 3, 4, \dots, k - 1$$

A cada número natural n le aplica:

```
n2while(n)
```

El resultado es el programa WHILE codificado por ese número. Por tanto, esta función enumera los primeros programas WHILE según el orden inducido por la codificación utilizada por la librería.

Esto refleja la idea teórica de que el conjunto de programas WHILE es enumerable. Es decir, existe una forma efectiva de listar todos los programas WHILE.

5. Código completo usado

El código completo empleado para resolver las actividades finales es el siguiente:

```
from python.whilelanguage import *
from python.whilelanguage.encoding import *

Q = """(3,
while X1 != 0 do
    X1 := X1 - 1;
```

```

        X4 := X4 + 1
    od;
while X2 != 0 do
    X2 := X2 - 1;
    X4 := X4 + 1
od;
while X3 != 0 do
    X3 := X3 - 1;
    X4 := X4 + 1
od;
X1 := X4)"""

print("TQ", t_steps(Q, [3, 5, 2]))
print("Res:", f_function(Q, [3, 5, 2]))

D = "X1:=X1+1; while X1!=0 do X1:=X1+1 od"

print(while2n(0, D))

def enumera_vectores(k):
    vectores = []

    for n in range(k):
        vectores.append(godeldecoding(n))

    return vectores

print(enumera_vectores(10))

def enumera_programas_while(k):
    programas = []

    for n in range(k):
        programas.append(n2while(n))

    return programas

programas = enumera_programas_while(10)

for i, programa in enumerate(programas):
    print(i, programa)

```

6. Conclusión

En esta práctica se ha comprobado cómo los programas WHILE pueden ejecutarse y analizarse paso a paso mediante Python. También se ha trabajado con la codificación de datos y programas mediante números naturales.

Las conclusiones principales son:

- Un programa WHILE puede calcular funciones numéricas parciales.
- Si un programa no termina, representa una función indefinida para esa entrada.
- La función `t_steps` permite calcular el tiempo de parada de un programa.
- La función `f_function` permite obtener el resultado calculado por el programa.
- Los vectores de $\mathbb{N}^*$ son enumerables mediante `godeldecoding`.
- Los programas WHILE son enumerables mediante `n2while`.
- La codificación permite relacionar programas, datos y números naturales, lo cual es fundamental para estudiar computabilidad.